EXPERIMENT - 14

Pro-Code Extension Using Custom Functions
Library Book Issue Management

Student Name	AJAY K
Register / Roll No.	241401006
Subject	Low-Code Application Development using Zoho Creator
Application	Library Book Issue Management
Platform	Zoho Creator
Record Date	[enter date]

Aim

To extend Library Book Issue Management with a custom function, validate its inputs, return a structured result and test normal, invalid and empty-result cases.

Experiment overview

Implementation and execution of a reusable, typed Deluge function that returns validated library queue statistics.

Department of Computer Science and Business Systems
Rajalakshmi Engineering College

241401006

1. Extension Choice

The syllabus permits Zoho Catalyst / custom functions. This experiment uses a custom Deluge function in Creator, keeping the calculation close to the request records and avoiding a separate service deployment.

2. Function Contract

Property	Definition
Function name	Get_Library_Queue_Summary
Namespace / language	Default / Deluge
Signature	map Get_Library_Queue_Summary(string status_name)
Allowed status values	Pending; Under Review; Available; Collected; Unavailable [match your own form's choices]
Success response	ok, status, request_count, copies_requested
Invalid response	ok = false and a readable error
Side effects	None: no data writes, emails or external API calls
Exposure	[State how you tested it, e.g. function editor only, no public API]

3. Implementation Procedure

1. Open Workflow > Functions and create a function returning map.
2. Add status_name as a string argument.
3. Normalize null/whitespace input and check an explicit status allow-list.
4. Use a filtered count and sum to obtain queue statistics.
5. Return zero copies for an empty queue.
6. Save and execute your test cases in Development. [Tick off only what you did.]

241401006

4. Deluge Source

Adjust form and field names (Book_Issue_Request_Form, Request_Status, Number_Of_Copies) to match your own application.

```
map Get_Library_Queue_Summary(string status_name)
{
 result = Map();
 allowed = {"Pending","Under Review","Available","Collected","Unavailable"};
 status_value = ifnull(status_name,"").trim();
 if(!allowed.contains(status_value))
 {
 result.put("ok",false);
 result.put("error","Use a valid library request status.");
 return result;
 }
 requests = Book_Issue_Request_Form[Request_Status == status_value].count();
 copies = 0;
 if(requests > 0)
 {
 copies = Book_Issue_Request_Form[Request_Status == status_value].sum(Number_Of_Copies);
 }
 result.put("ok",true);
 result.put("status",status_value);
 result.put("request_count",requests);
 result.put("copies_requested",copies);
 return result;
}
```

5. Reuse Example

```
summary = thisapp.Get_Library_Queue_Summary("Pending");
info summary;
```

A later workflow or page can call this function to reuse the calculation. [State whether yours was tested only in the function editor or wired into a dashboard or published to Production.]

6. Test Cases and Actual Outputs

Run each test in your own app and record your real output.

Test input	Observed output	Result
Pending	[your output]	[Pass / Fail]
InvalidStatus	[your output]	[Pass / Fail]
Collected (or another empty status)	[your output]	[Pass / Fail]

7. Test Interpretation

[Explain your results using your own numbers. Note that an invalid status should return a clear error rather than an empty queue, and a valid empty queue should return zeros. State which cases you did not execute.]

8. Security and Maintenance

Only a fixed set of statuses is accepted. The result includes aggregate values and does not return member names, addresses or contact details. Keep invocation behind appropriate application permissions if it is later exposed through a page or API. Retest the allow-list when status choices change.

241401006

Implementation Evidence

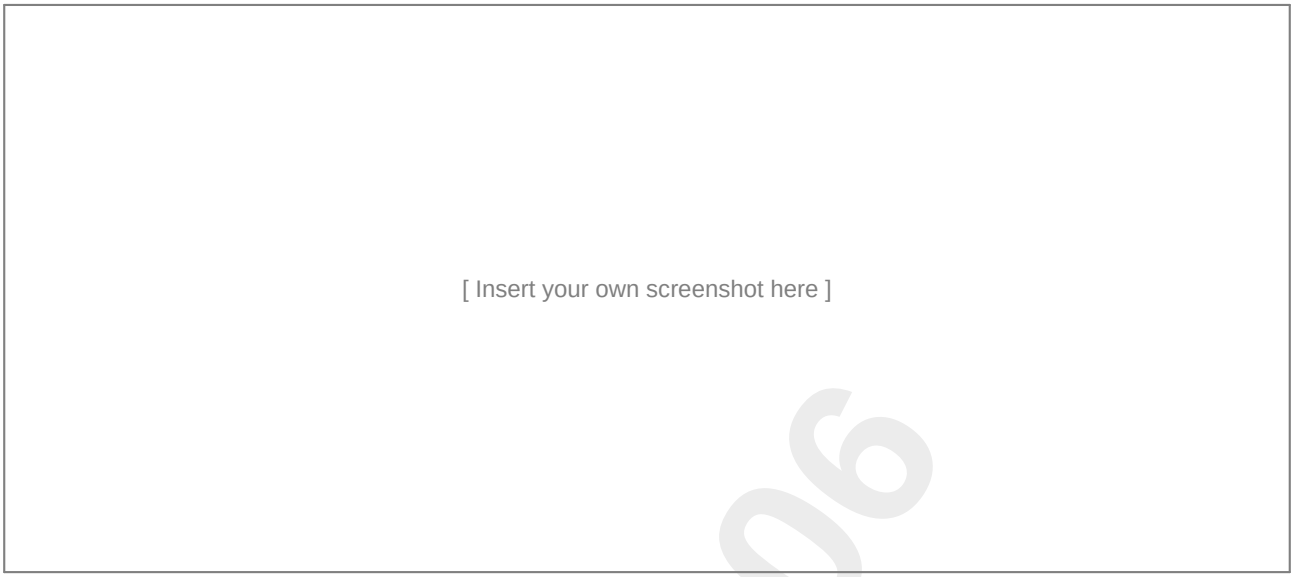

Figure 14.1. Development execution for Pending.

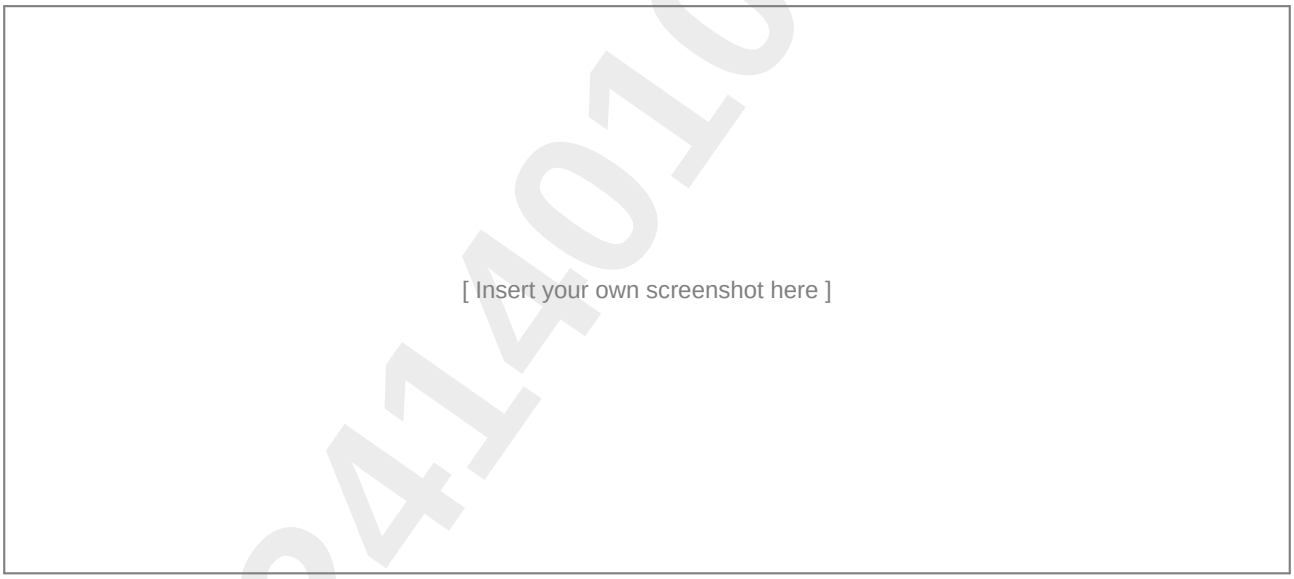

Figure 14.2. Development execution for InvalidStatus.

241401006

Implementation Evidence

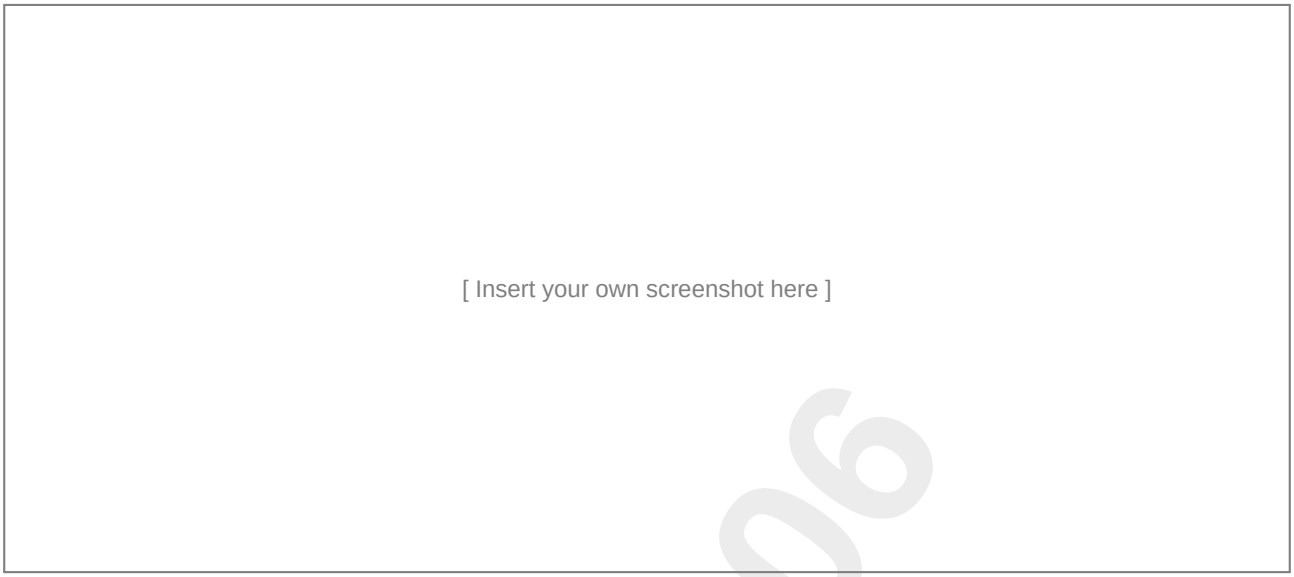

Figure 14.3. Development execution for an empty valid queue.

241401006

9. Catalyst Comparison

Custom function used here	Possible Catalyst extension
Runs inside Creator and reads the request form directly.	Would expose a defined service interface and require authorization.
Suitable for a small queue summary.	Could host a specialized computation or reservation service.
No separate hosting setup for this lab.	Requires a project, deployment and operational monitoring.

10. Result

[State what you implemented and how many of your test cases passed.] The function demonstrates how typed custom logic can extend a low-code application without rewriting the existing form and workflow. Catalyst was studied as an alternative; no Catalyst project or cloud function was deployed.

References

1. Understanding Creator functions
2. Catalyst serverless introduction

241401006